

CHAPTER 1: INTRODUCTION

Automated fruit sorting represents a critical challenge in the agricultural sector, driven by the need for increased efficiency, consistent product quality, and the reduction of labor dependency. Traditional methods of fruit sorting are labor-intensive, often requiring large teams of workers to manually inspect and categorize fruits based on their , color, , and quality. These manual processes are prone to human error, leading to inconsistencies in sorting and quality control, which can result in a significant economic impact. Moreover, with the ongoing labor shortages and rising costs of labor in agriculture, there is increasing pressure to find automated solutions that can handle these tasks more effectively and at scale. While the use of machine learning (ML) and computer vision in automated fruit sorting offers numerous benefits, there are several challenges and complexities that must be addressed to fully realize it's potential. Understanding these challenges in greater detail helps to provide a more comprehensive view of the problem and the solutions required to tackle them effectively. Fruits are highly variable in shape, , color, and texture, even within the same variety. This natural variability can pose significant challenges for automated systems, particularly in the AI and ML are transforming the fruit sorting industry by making processes more accurate, efficient, and scalable. By automating fruit classification, quality detection, and predictive maintenance, these technologies enhance productivity and reduce costs. The future of fruit sorting systems lies in the continued advancement of AI and ML, leading to more intelligent and adaptable systems that can handle a wide variety of fruits and operational challenges context of machine learning.



Fig 1.1 : Prototype of fruit segregation

CHAPTER2: LITERATURE REVIEW

2.1 NEED OF IDENTIFICATION OF FRUIT

2.1.1. Automation of Sorting Process

Manual sorting of fruits is labor-intensive and time-consuming. By automating the identification process, fruit sorting systems can work continuously at high speeds, significantly increasing throughput. Automation ensures that large quantities of fruits can be processed without human intervention, leading to greater efficiency.

Benefit: Faster processing of large batches of fruit, with minimal human error.

2.1.2 Accurate Classification and Grading

Identifying different types of fruits and their characteristics, such as , shape, color, and texture, allows the system to classify them accurately. This is essential for sorting fruits based on various criteria like , quality, and , which is critical for both consumer satisfaction and market standards.

Benefit: Consistency in sorting, ensuring that only high-quality fruits reach consumers, and subpar fruits are removed.

2.1.3. Customization for Different Fruits

Each type of fruit has its own unique characteristics, which require specific sorting parameters. AI-powered identification systems can be trained to recognize different fruits (apples, oranges, bananas, etc.) and sort them accordingly. The system can adapt to different types of fruit, even in mixed batches.

Benefit: Flexibility in handling a variety of fruits without the need for reprogramming or manual adjustments.

2.2 REAL-TIME MONITORING AND FEEDBACK

Real-time monitoring and feedback refer to the continuous observation, analysis, and adjustment of processes or systems as they occur. This concept is widely used in various fields, including industrial automation, manufacturing, healthcare, and even software systems. The goal is to gather data in real-time, analyze it instantly, and take immediate action or provide feedback to users, operators, or systems. In the context of a fruit sorting system (or any automated system), real-time monitoring and feedback help in continuously assessing and optimizing the system's performance, ensuring efficiency, accuracy, and quality control.

Key Components of Real-Time Monitoring and Feedback

1. Data Collection (Monitoring): Gathering real-time data from sensors, cameras .
2. Data Processing: Analyzing the collected data to derive insights and identify any issues or patterns.
3. Feedback: Communicating the results of the analysis back to the system, users, or operators for immediate actions.
4. Actionable Insights: Taking immediate actions based on the feedback to optimize or correct the process.

AI-based systems can continuously monitor fruit quality in real-time, providing valuable feedback to operators or producers. This can help in adjusting sorting criteria or identifying trends in fruit quality that may require intervention or improvement.

Benefit: Real-time insights that help in improving the sorting process and preventing potential issues before they escalate.

2.3 2D RGB CAMERA

These cameras capture visual images in Red, Green, and Blue (RGB) color channels. They provide two-dimensional images, which are helpful in basic fruit classification tasks based on color, shape, and They are typically used for recognizing fruit characteristics such as (color), , and general shape, as these features are crucial for sorting.

Basic Fruit Classification: A 2D RGB camera can help in classifying fruits by identifying basic visual attributes such as:

- Size: Determining the of fruits to segregate them into different categories (e.g., small, medium, large).
- Shape: Classifying fruits based on their shape (e.g., round for apples or elongated for bananas).

2.4 ROBOTIC ARMS AND GRIPPERS IN FRUIT SORTING SYSTEMS

Function of Robotic Arms and Grippers:

In fruit sorting systems, robotic arms equipped with grippers are used to handle, sort, and position fruits efficiently on a conveyor belt. These robotic systems are integral in automating the process of picking, placing, and packaging fruits after they have been identified and classified by cameras or sensor.

2.4.1. Robotic Arms

Role and Function:

- Movement and Precision: Robotic arms are designed to move in multiple directions and are equipped with high precision to pick up, manipulate, and place fruits. They replicate human arm movements with great accuracy, which is essential for delicate tasks like handling fruits without causing damage.
- Automated Sorting: After fruits are identified and classified based on , color, and quality (via cameras and sensors), the robotic arm can pick up each fruit and place it into the appropriate sorting bin or packaging containers.

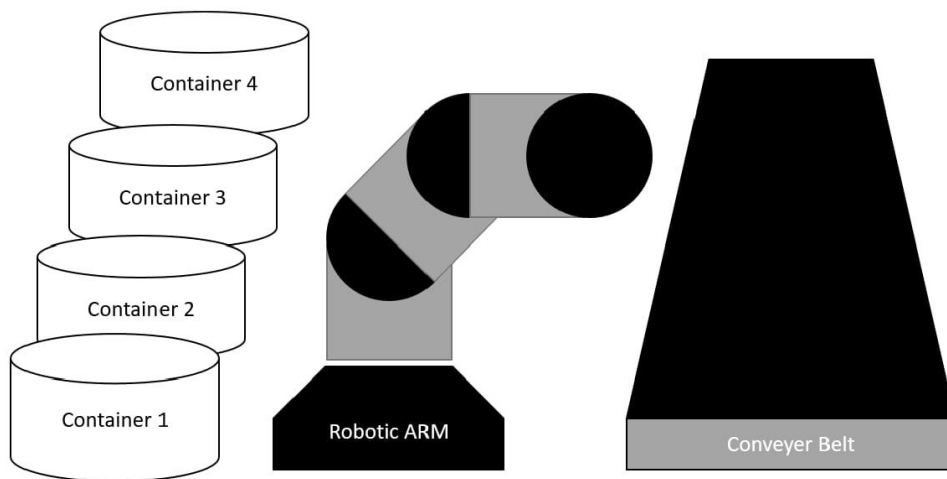


Fig 2.1: Robotic Arm

2.4.2. Grippers

Role and Function:

Grippers are attached to the end of robotic arms and are responsible for physically gripping and holding onto fruits during the sorting process. The design of the gripper must be chosen based on the fruit's , shape, and fragility to avoid damaging it.



Fig 2.2: Grippers

2.5 SORTING MECHANISMS OF CONVEYOR BELTS IN FRUIT SORTING SYSTEMS

Conveyor belts are central to many automated sorting systems, especially in industries like agriculture, where high-speed sorting and categorizing of products (such as fruits) are required. Sorting mechanisms on conveyor belts help automate the process of separating and directing fruits into various categories based on predetermined criteria like , shape, color, , or quality. Here's an overview of how sorting mechanisms on conveyor belts work:

Basic Conveyor Belt Sorting Mechanism

In a basic sorting system, fruits move along a conveyor belt, and sorting decisions are made based on physical characteristics such as:

1. Size
2. Shape
3. Color

Typically, after fruits pass through a series of visual and mechanical checks (like cameras, sensors, and AI), sorting mechanisms are triggered to separate fruits into designated categories.

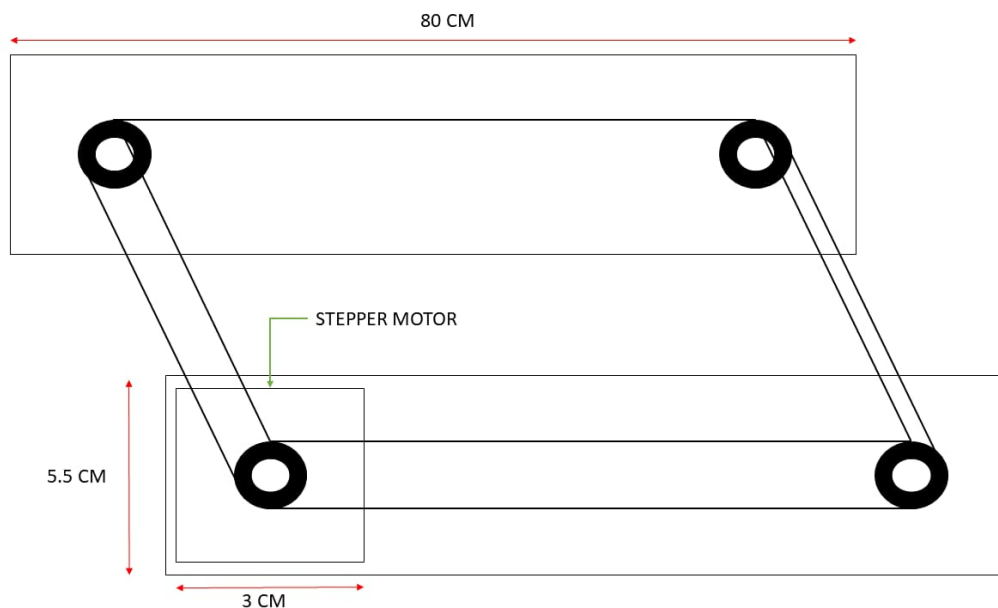


Fig 2.3: Conveyor Belt

2.6 MACHINE LEARNING AND SOFTWARE ALGORITHMS

2.6.1. ROLE OF MACHINE LEARNING IN FRUIT SORTING SYSTEMS

Machine learning in fruit sorting systems involves using data-driven techniques to classify and analyze fruits based on various features like color, shape, and quality. With ML, the system learns from the data over time and improves its performance as more examples are provided.

Fruit Classification

Goal: Classify fruits into categories like type (e.g., apples, bananas, oranges), size (e.g., small, medium, large)

- How it Works:

Supervised Learning: A model is trained using labeled data (i.e., fruit images with known categories). For instance, an image of an apple with the label “apple” is used to teach the model how to recognize apples.

The model learns patterns from the data and can later predict the type, size, or quality of unseen fruits in real-time.

2.6.2 SOFTWARE ALGORITHMS FOR FRUIT SORTING SYSTEMS

Software algorithms control the overall operation of the sorting system, including how data from sensors and cameras are processed, how fruits are classified, and how actions (like sorting or rejecting fruits) are taken.

1) Image Processing Algorithms

Purpose: Enhance and process images from cameras to extract meaningful features such as shape, color, and texture.

Common Techniques:

- Edge Detection: Helps detect the boundaries of fruits, especially if they are not well-separated on the conveyor belt. Algorithms like Canny edge detection can be used to find the edges of the fruit and segment them from the background.
- Contour Detection: Used to outline the shape of an object (fruit) to assess its size and shape. This is important for sorting based on size or type.

2) Decision-Making Algorithms

Purpose: Make sorting decisions based on the classification and feature data provided by ML models or image processing algorithms.

How it Works:

Once fruits are classified by machine learning models, decision-making algorithms (e.g., rule-based systems, decision trees) decide what action should be taken (e.g., sorting a fruit to a bin, rejecting it for defects).

2.7 CALIBRATION AND TESTING TOOLS

Types:

- Calibration Kits (for Cameras, Sensors)
- Test Platforms for System Validation

Function:

- Calibration kits are used to ensure that cameras and sensors are properly aligned and configured for optimal image capture.
- Test platforms allow operators to simulate real-world conditions and validate the system's accuracy before full deployment.

2.8 KEY ADVANCEMENTS AND METHODS USED

2.8.1. The Role of Computer Vision and Image Processing

The emergence of computer vision techniques in the 1990s brought new possibilities to fruit sorting. Early approaches leveraged basic image processing algorithms such as edge detection, color analysis, and texture recognition to classify fruits (Gonzalez & Woods, 2008). These methods allowed for the automated detection of fruit type, , and color, but still struggled with complex tasks like defect detection or classification under varied lighting conditions.

2.8.2. Machine Learning and Deep Learning in Fruit Sorting

YOLOv3 (You Only Look Once): YOLOv3 is a real-time object detection system that has been widely used for fruit sorting due to its speed and accuracy in identifying and classifying multiple objects in an image. Research by Redmon et al. (2016) introduced YOLOv3 as a fast, real-time object detection algorithm capable of classifying fruits in high-speed sorting environments. YOLOv3 can identify and classify multiple fruit types simultaneously, which is critical for high-throughput systems.

One of the primary challenges in developing machine learning models for fruit sorting is the need for large, labeled datasets. Collecting thousands of labeled images of different fruits under varied lighting conditions, angles, and environments is both time-consuming and expensive. Data augmentation techniques—such as rotation, flipping, and scaling—are often used to artificially expand the of the dataset .

Crowdsourcing is another method to overcome this challenge. Platforms like Amazon Mechanical Turk have been used to gather labeled data for fruit classification tasks, enabling faster dataset creation and improving model robustness

2.8.3. Real-Time Processing and Industrial Applications

Real-time fruit sorting is essential for large-scale agricultural operations, where the sorting process must keep pace with the flow of fruits through a conveyor belt or packing line. Several studies have focused on optimizing deep learning models for real-time processing in industrial environments. For example, Yoon et al. (2020) demonstrated the use of real-time -based systems that classify fruits in less than 100 milliseconds, meeting the demands of high-throughput fruit sorting in commercial packing lines.

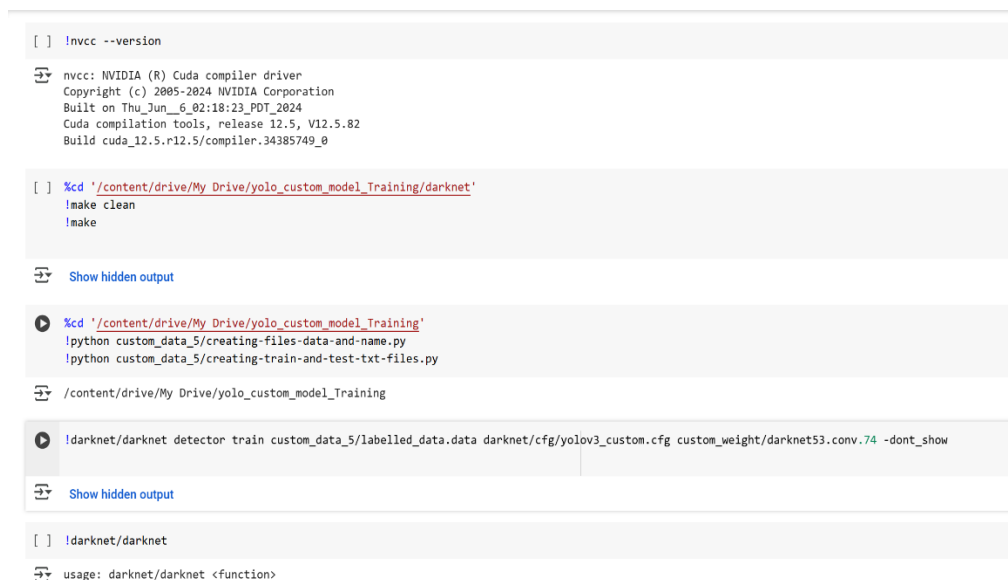
The use of edge computing—where the processing is done closer to the data source (i.e., at the sorting station)—has also gained attention as a way to reduce latency and improve the speed of real-time fruit detection systems.

2.9 IMPLEMENTATION OF SOFTWARE

OpenCV is a powerful library in Python used for real-time computer vision tasks such as image processing, object detection, and classification. For an automated fruit segregation system, OpenCV can be used to process images from cameras and sensors to: Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.

- Pre-process images: For example, converting images to grayscale, applying blurring, edge detection, or thresholding, which are important steps before analyzing fruit features (e.g., shape, color).
- Detect and classify fruits: Using techniques like contour detection, color segmentation, and feature matching to identify different fruits or defects.
- Apply machine learning models: OpenCV can interface with machine learning models (like those built with TensorFlow or PyTorch) to recognize more complex patterns and classify fruits accurately. Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.

Google Colab is an online platform that allows you to write and execute Python code in an interactive notebook environment. It provides several advantages for developing, testing, and training vision systems like the one used in fruit segregation:



```
[ ] !nvcc --version

nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Jun__6_02:18:23_PDT_2024
Cuda compilation tools, release 12.5, V12.5.82
Build cuda_12.5.r12.5/compiler.34385749_0

[ ] %cd '/content/drive/My Drive/yolo_custom_model_Training/darknet'
!make clean
!make

Show hidden output

%cd '/content/drive/My Drive/yolo_custom_model_Training'
!python custom_data_5/creating-files-data-and-name.py
!python custom_data_5/creating-train-and-test-txt-files.py

/content/drive/My Drive/yolo_custom_model_Training

!darknet/darknet detector train custom_data_5/labelled_data.data darknet/cfg/yolov3_custom.cfg custom_weight/darknet53.conv.74 -dont_show

Show hidden output

[ ] !darknet/darknet

usage: darknet/darknet <function>
```

Fig 2.4:Google Colab

Advantages of Using Google Colab:

- **Easy Setup:** Google Colab comes pre-configured with many libraries, including OpenCV, NumPy, TensorFlow, and PyTorch, which are essential for building computer vision models. Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.
- **Collaboration:** Being cloud-based, Colab allows easy sharing of notebooks with team members for collaboration on the development of the fruit sorting system.
- **Integration with Google Drive:** You can store datasets and models on Google Drive and access them directly from Colab notebooks.

2.10 COST ESTIMATION

a) Current Components Cost Estimation:

Component : Approximate Cost (INR)

USB Webcam: 800

Arduino Nano :400

Servo Robotic Arm: 2,000

Fixed Conveyor: 5,000

PC + Arduino: 25,000

Wooden and Iron Frame: 2,000

Total Cost: INR 35,200

b) large scale integration

Component: Approximate Cost (INR)

Industrial Camera:15,000

PLC Controller:8,000

Air Suction Robotic Arm: 45,000

Grid Line Conveyor Belt: 20,000

Edge AI Device: 30,000

Smart IoT Gateway:5,000

Stainless Steel Frame: 10,000

Total Cost: INR 133,000

CHAPTER3: DESIGN CONSIDERATIONS AND CHALLENGES FOR FRUIT SORTING SYSTEM DEVELOPMENT

Designing and developing an automated fruit sorting system involves numerous considerations and challenges. These factors must be addressed to ensure the system is efficient, accurate, and capable of handling a variety of fruits. Below are key design considerations and the challenges that developers and engineers must overcome during system development.

3.1 DESIGN CONSIDERATIONS

3.1.1 Fruit Variety and Characteristics

Consideration: Fruits come in various shapes, colors, and textures. For example, apples, bananas, oranges, and grapes have distinct features that require specific sorting approaches.

Solution: The system must be capable of handling multiple types of fruits, often simultaneously. This can be achieved by incorporating flexible algorithms (e.g., machine learning models like s) that can adapt to different types of fruit, and multi-sensor setups to capture various features (e.g., color, shape, , and texture).

3.1.2 Speed and Throughput

What are CUDA and cuDNN?

CUDA (Compute Unified Device Architecture) is a groundbreaking platform for parallel computing created by NVIDIA. It provides programmers and researchers direct access to NVIDIA GPUs' virtual instruction set. CUDA improves the efficiency of complex operations such as training AI models, processing large datasets, and conducting scientific simulations.

cuDNN (CUDA Deep Neural Network library) is a specialized, GPU-accelerated library that provides essential building blocks for deep neural networks. It's designed to deliver high-performance components for convolutional neural networks, recurrent neural networks (RNNs), and other complex deep learning algorithms. By implementing cuDNN, frameworks such as TensorFlow and PyTorch can take advantage of optimized GPU performance.

In short, NVIDIA's CUDA installation lays the groundwork for GPU computing, whereas cuDNN provides targeted resources for deep learning. This combination enables remarkable GPU acceleration for tasks that a traditional CPU could otherwise require days or weeks to complete.

Consideration: The sorting system needs to operate at high speeds for efficient processing, especially in large-scale operations. Sorting times must be optimized to avoid bottlenecks.

Solution: Designing a system with high-speed conveyor belts, fast image processing algorithms, and quick-actuating sorting mechanisms (e.g., air jets, robotic arms) is critical. The system should also be capable of handling large volumes of fruits in real-time.

3.1.3 Accuracy and Precision

Consideration: The system must accurately sort fruits based on specific parameters (, shape, color, quality). Mis-sorting can lead to wasted fruits, lower product quality, and increased operational costs.

Solution: we are using f1 score as a confidence of less than 5.

3.1.4 Robustness and Durability

Consideration: The sorting system will be exposed to continuous wear and tear, dust, moisture, and temperature variations. It must be robust enough to withstand these conditions.

Solution: The system should be designed using durable materials (e.g., stainless steel) and weather-resistant components. Sealed electrical components and waterproof sensors are essential to prevent damage from moisture or environmental factors.

3.1.5 Scalability

Consideration: The system should be scalable, meaning it should be able to handle increasing volumes of fruits as the business grows.

Solution: A modular design approach allows for easy scaling. The system can incorporate additional conveyor belts, more sorting bins, or extra robotic arms without disrupting operations.

3.1.6 Cost-Effectiveness

Consideration: The development and maintenance costs of the system should be balanced against the potential return on investment. The system must provide value through improved sorting speed, quality control, and operational efficiency.

Solution: Use cost-effective technologies like open-source software (e.g., OpenCV, TensorFlow) and affordable hardware (e.g., industrial cameras, basic sensors) without sacrificing performance. Additionally, employing modular components allows easy upgrades and reduces long-term costs. Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.

3.2 CHALLENGES IN SYSTEM DEVELOPMENT

3.2.1 Variability in Fruit Characteristics

Challenge: Fruits are inherently variable in shape, , texture, and color. For example, an apple's color might vary from green to red depending on , and the shape can vary slightly.

Solution: Implementing robust image processing techniques combined with machine learning algorithms (e.g., s) to detect subtle variations in fruit characteristics. The system should be trained with a large dataset of different fruit types, varieties, and quality levels to ensure high generalization.

3.2.2 Real-Time Processing and Low Latency

Challenge: Sorting must happen in real-time without delays, as the fruits continuously move on the conveyor belt. Processing the data from multiple sensors (cameras, weight sensors, etc.) quickly can lead to latency issues.

Solution: Use high-performance computing platforms (e.g., GPUs or specialized processing units) to accelerate image processing and inference tasks. Optimization techniques like model pruning and quantization can reduce the computational load while maintaining accuracy. Edge computing solutions, where processing happens close to the sensors, can further reduce latency.

3.2.3 Illumination and Environmental Conditions

Challenge: Lighting conditions, such as shadows, reflections, or inconsistent lighting on the conveyor belt, can significantly affect the accuracy of image processing algorithms.

Solution: Use controlled, consistent lighting (e.g., LED arrays) to ensure uniform illumination across the fruits. Additionally, image preprocessing techniques (e.g., histogram equalization) can help normalize lighting conditions in captured images. The system can also use multiple cameras to mitigate the effect of poor lighting.

3.2.4 Hardware Integration and Synchronization

Challenge: Integrating sensors, cameras, actuators (e.g., robotic arms, diverters), and sorting mechanisms in a synchronized manner is a complex task. Ensuring smooth communication between software and hardware components is crucial.

Solution: Employ industrial communication protocols (e.g., Modbus, EtherCAT) and real-time control systems to synchronize actions between different components. Feedback loops and closed-loop control systems help monitor and adjust actions in real-time.

3.2.5 Fruit Handling and Damage Prevention

Challenge: Delicate fruits (e.g., peaches, tomatoes) are prone to bruising or damage during sorting. Handling them with the correct force is crucial to avoid causing harm.

Solution: Use soft robotic grippers, vacuum-based picking systems, or gentle conveyors that apply minimal force while handling delicate fruits. The sorting system should also incorporate real-time monitoring to adjust the gripping or sorting technique based on the fruit type.

3.2.6 Data Quality and Labeling for Training Models

Challenge: Training machine learning models requires high-quality, labeled data. Collecting and labeling data for different types of fruits and defects can be time-consuming and expensive.

Solution: Crowdsourcing or leveraging existing datasets can help gather a large amount of labeled data. Additionally, using data augmentation techniques can artificially increase the size of the dataset by creating variations of existing data through transformations (e.g., rotating, scaling images).

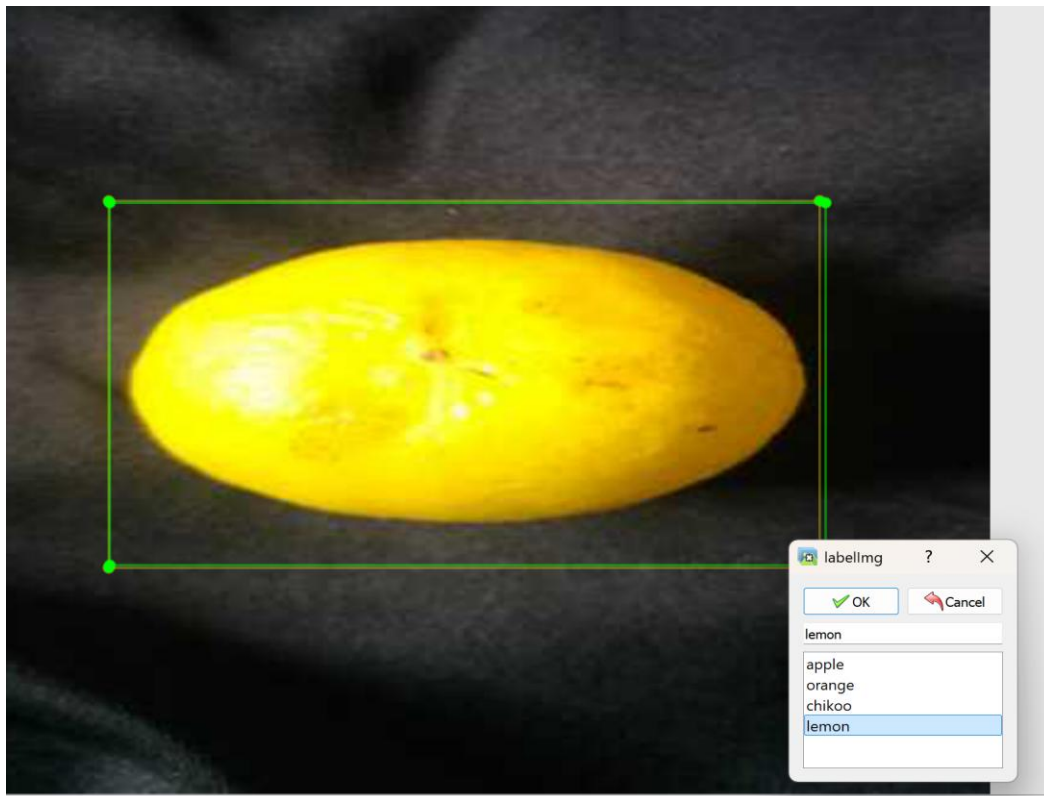


Fig 3.1:Lemon detection



Fig 3.2:Apple detection

3.2.7 Maintenance and Upkeep

Challenge: The system will require regular maintenance to ensure the mechanical parts (conveyor belts, robotic arms) and sensors (cameras, weight sensors) continue to operate efficiently. Failure to maintain the system can lead to errors or downtime.

Solution: Incorporate predictive maintenance algorithms that can monitor the health of key components and predict when maintenance is required. This helps minimize downtime and maintain system reliability.

3.3 CONCLUSION

Designing a fruit sorting system that integrates machine learning, software algorithms, and hardware components is a complex task that requires addressing various design considerations and challenges. To ensure success, system developers must prioritize:

- Scalability and flexibility to handle different fruit types and growing volumes.
- Accuracy in classification and defect detection to avoid mis-sorting and waste.
- Real-time performance to avoid delays in the sorting process.
- Durability and maintenance considerations to ensure the system can withstand continuous use in potentially harsh environments.

CHAPTER 4: HARDWARE IMPLEMENTATION

A Robotic Fruit Segregation System is designed to automatically identify, classify, and separate different types of fruits based on color, shape or other parameters using computer vision, sensors, and robotic mechanisms. The system typically integrates machine learning (YOLOv3, s), robotic arms, conveyor belts, and microcontrollers (Arduino, , etc.).

Block Diagram of the System

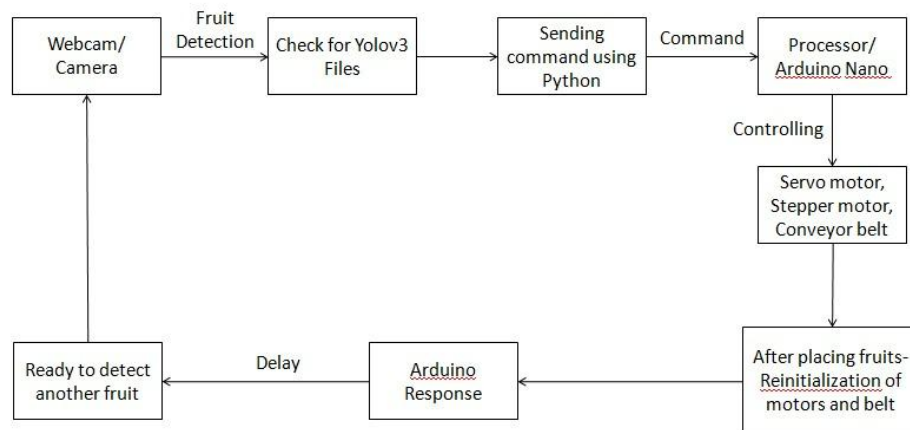


Fig 4.1: Block Diagram

4.1 SYSTEM WORKING

4.1.1 Input & Data Acquisition

- The system captures real-time images/videos of fruits using a camera mounted above a conveyor belt.

4.1.2 Image Processing & Object Detection

- The camera captures images, which are processed using OpenCV and Deep Learning (YOLOv3, YOLOv5). Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.
- The YOLOv3 model detects fruits in the frame and assigns bounding boxes with confidence scores.
- Fruits are identified based on class IDs from a trained dataset (e.g., apples, oranges, lemons, bananas).

Steps in Image Processing:

- Preprocessing: Convert image to grayscale or HSV color space for better fruit identification. Re image and remove noise.
- Feature Extraction: Detect features like color, texture, shape, and . Extract the Region of Interest (ROI) of the detected fruit.
- Example:

Red → Apple

Yellow → Lemon

Orange → Orange

Shape Analysis:

- Detect round, oval, or irregular shapes to confirm fruit type.
- Size Estimation: Determine the diameter of the fruit for segregation into small/large categories. yolov3 neglects the size.

4.1.3 Robotic Mechanism & Fruit Segregation

1. A robotic arm (with suction cups, grippers, or vacuum-based pickers) or mechanical flaps segregates the fruits based on classification.
2. Different techniques for segregation: Robotic Arm Pick & Place: Picks fruit and places it in a category-specific bin.
3. Conveyor Belt Sorting: Fruits are pushed into bins using actuators.
4. Air Jet Sorting: High-pressure air pushes lighter fruits into separate bins.

4.1.4 Communication with Microcontroller (Arduino)

The system sends signals to Arduino to execute actions like: Activating motors for robotic arm movement. Controlling conveyor belts to direct fruit to the correct bin. Sending alerts or feedback to a control panel. Arduino/Microcontroller processes commands and executes sorting actions.

4.1.5 Output & Storage

- The system displays real-time classification results on a monitor.
 - Data (e.g., number of apples, oranges, defective fruits) is stored for analysis.
 - IoT Integration: Sends data to cloud storage for tracking and optimization.
 - Automated Rejection: Defective/damaged fruits are removed automatically.
1. Workflow:
 1. Camera captures fruit image.
 2. YOLOv3 model detects fruit type and color.
 3. Decision-making unit classifies fruit (e.g., apple, lemon, or orange).
 4. Signal sent to Arduino/ for robotic action.
 5. Robotic arm picks and places fruit in the correct bin.
 6. System updates inventory and stores data.

4.2 DESIGN CONSIDERATIONS AND UPCOMING AMENDMENTS

Developing a robotic system for fruit segregation involves several key design considerations and potential future improvements to enhance efficiency, accuracy, and overall performance. Below are the critical factors to consider and the anticipated amendments for optimizing the system:

Mechanical Design:

4.2.1 End-effector/Gripper Design

- Soft, adaptive grippers to handle fruits of varying size and shapes without causing damage.
- At large scale industries Vacuum-based or finger-based grippers with force feedback sensors for gentle handling.

4.2.2 Conveyor and sorting mechanism

- Adjustable conveyor belt speeds to match throughput requirements.
- Anti-slip and food-grade conveyor materials to ensure safety and hygiene.

4.2.3 Mobility

- Multi-axis robotic arms for better reach and maneuverability.
- Precision actuators for quick and accurate movement.

4.2.4. Vision and Sensing System

a) Camera and imaging technology

- High-resolution RGB and depth cameras for accurate shape, color, and detection.
- Multi-angle imaging to reduce blind spots.

b) Spectral analysis

- Near-Infrared (NIR) and Hyperspectral imaging to assess internal quality (e.g., sugar content, bruising).

c) Real-Time Data Processing

- High-speed processors and low-latency communication protocols.
- Cloud-based data analysis for predictive maintenance and performance tracking.

d) Safety and Compliance

- Use of food-grade materials for all contact surfaces.
- Compliance with FDA and HACCP (Hazard Analysis and Critical Control Points) standards.

e) Safety for Human Interaction

- Emergency stop buttons and safety barriers.
- Collaborative robot (cobot) design with force-limiting joints.

4.2.5 Energy Efficiency and Sustainability

- **Power Consumption**
 - Use of energy-efficient motors and actuators.
 - Regenerative braking to conserve energy.
- **Material Selection**
 - Lightweight, durable materials to reduce overall power demand.
 - Recyclable components to minimize environmental impact.

4.2.6 Mechanical Enhancements

- **Adaptive gripping**
 - Introduction of smart, self-adjusting grippers to handle softer fruits without damage.
 - Integration of tactile sensors to measure fruit firmness and adjust grip pressure dynamically.
- **Modular Design**
 - Designing a modular robot arm and conveyor system for easy maintenance and scalability.
 - Quick swap-out mechanisms for faster maintenance and downtime reduction.

4.2.7 Networking and Integration

- **IoT Integration**
 - Real-time tracking and reporting of sorting performance through cloud connectivity.
 - Remote diagnostics and over-the-air (OTA) software updates.
- **Smart Farm Integration**
 - Linking the robot to farm data for predictive sorting based on harvest conditions.
 - Autonomous coordination with other agricultural machines (e.g., harvesters, packing lines).
- **Human-Machine Collaboration**

User-Friendly Interface:

- Intuitive touchscreen controls with multi-language support.
- Customizable sorting criteria based on fruit type, customer order, or market demand.
- Machine learning to adjust sorting behavior based on operator feedback.
- Operator training modules with AR/VR-based simulations.

Conclusion

Enhancing a robotic fruit segregation system requires a balance of robust mechanical design, advanced machine learning algorithms, and intelligent control systems. Upcoming amendments will focus on increasing adaptability, improving defect detection.

4.3 COMPONENTS USED

4.3.1 MG996R METAL GEAR SERVO MOTOR:

The **MG996R** is a popular metal gear servo motor widely used in various robotics, RC (radio-controlled) vehicles, and automation projects. Below are the key details of the MG996R servo motor:



Fig 4.2: Servo Motor

Key Specifications:

1. **Motor Type:** DC motor
2. **Operating Voltage:** 4.8V to 7.2V (commonly used with 5V or 6V power supplies)
3. **Torque:**
 - At 4.8V: 9.4 kg·cm
 - At 6.0V: 11 kg·cm
4. **Speed:**
 - At 4.8V: 0.19 sec/60°
 - At 6.0V: 0.15 sec/60°
5. **Rotation Angle:** 180° (standard rotation range)
6. **Control Signal:** PWM (Pulse Width Modulation) signal
7. **Gear Type:** Metal gears (this is one of the main features that make it stronger and more durable compared to plastic-gear servos)
8. **Dimensions:** Length: 40.7 mm

- Width: 19.7 mm
- Height: 42.9 mm

9. Weight: Approximately 55 grams

10. Connector: 3-pin (Power, Ground, Signal)

11. Temperature Range: -30°C to 60°C (operational temperature range)

12. Rotation Type: Standard 180° rotation (but some applications may require modifications for continuous rotation)

Features:

- **High Torque:** The MG996R is designed to provide high torque at a relatively low cost, making it ideal for many medium-d robot projects and RC models.
- **Metal Gears:** These gears provide durability, enabling the servo to handle larger loads and reducing the risk of stripping or breaking under heavy use.
- **Wide Voltage Range:** It operates between 4.8V and 7.2V, allowing for flexibility depending on your power source.
- **PWM Control:** The motor uses a PWM signal to determine the angle, allowing for precise control.

4.3.2 ARDUINO NANO EXPANSION SHIELD:

The **Arduino Nano Expansion Shield** is designed to expand the capabilities of the **Arduino Nano** microcontroller board, allowing you to connect additional hardware modules and components that aren't directly supported by the Nano's limited pins or I/O capabilities. The expansion shield provides more flexibility, a larger number of pins, and often includes additional features like motor drivers, communication interfaces, or sensor connections.

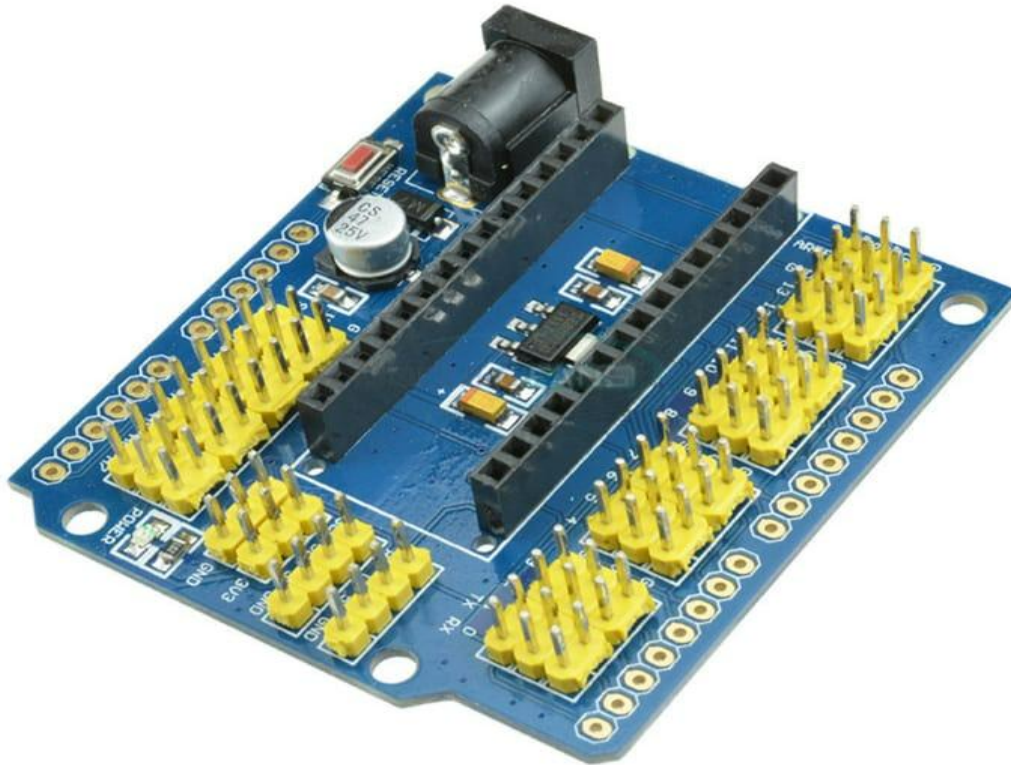


Fig 4.3: ARDUINO NANO EXPANSION SHIELD

Key Features:

1.Pin Extension:

- The shield provides additional headers that allow you to access more pins from the Arduino Nano.
- Typically, you will find extended **digital I/O pins**, **analog inputs**, and **power pins**.

2.Motor Driver Capabilities:

- Many expansion shields come with built-in **motor driver ICs** (e.g., L298N or L293D), which allow you to control motors (DC motors, stepper motors, or servo motors).

3. I2C / SPI Interfaces:

- Some shields come with built-in support for communication protocols like I2C or SPI, which can be used for communication with sensors, displays.

4.Power Supply:

- The shield often includes a dedicated **power supply section** that can provide a stable power source to the Arduino Nano and any connected peripherals. You may also have options to connect **external power** sources for motors or high-power components.

5.Breadboard Compatibility:

- The shield may be designed to fit on top of a **breadboard** for prototyping and testing, making it easy to add external components.

6. Onboard Sensors:

- Some expansion shields may include built-in sensors, such as **temperature sensors**, **humidity sensors**, or **accelerometers**, which can be directly interfaced with the Nano.

7.Arduino Nano Compatibility:

The shield is specifically designed to be compatible with the **Arduino Nano** board, which means it fits onto the Nano's header pins.

Common Types of Nano Expansion Shields:

1. **Motor Driver Shields:** For controlling motors, including stepper motors or DC motors.
2. **Sensor Shields:** To connect a variety of sensors such as temperature, light, or pressure sensors.
3. **Communication Shields:** To add **Wi-Fi**, **Bluetooth**, or **Ethernet** connectivity to your Nano.
4. **Relay Shields:** For controlling high-voltage devices like lights, fans, or other appliances.

Benefits:

- **Convenience:** Expands the functionality of your Arduino Nano with little effort.
- **Versatility:** Great for projects requiring multiple I/O pins or communication with external devices.
- **Prototyping:** Makes it easier to create and test complex circuits by offering convenient pin access and extra features.

4.3.3 ARDUINO NANO BOARD:

The **Arduino Nano** is a small, compact microcontroller board that is designed for use in a variety of electronics projects. It is part of the **Arduino family**, which is well-known for being beginner-friendly and versatile for prototyping and experimentation. Below are the key details about the **Arduino Nano board**:

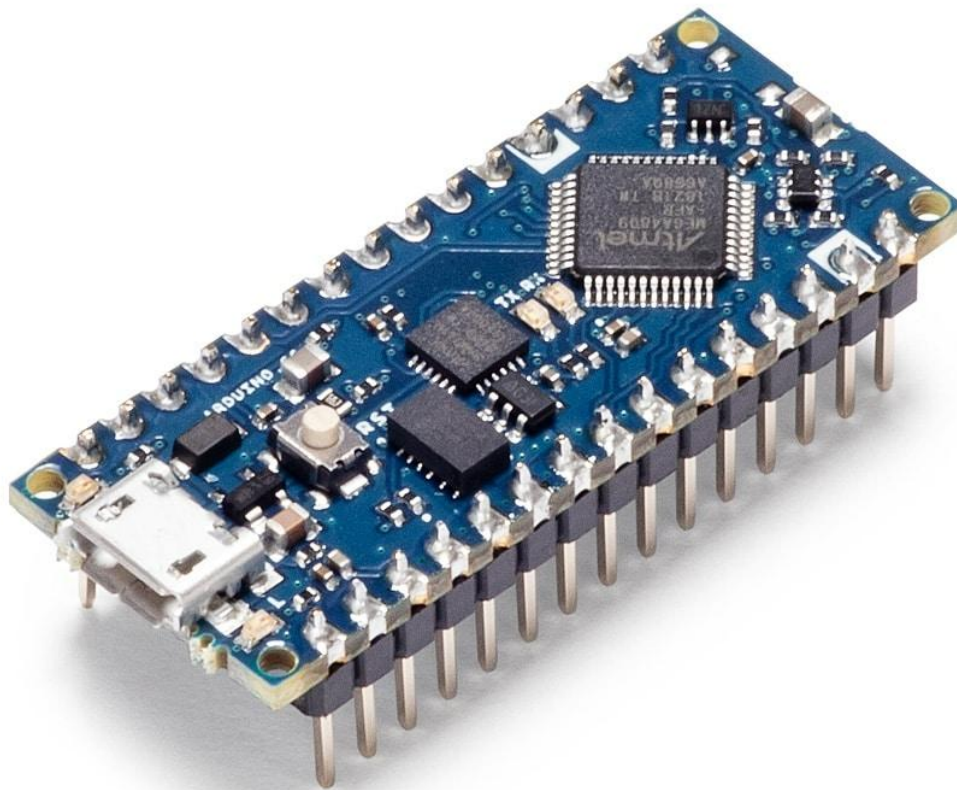


Fig 4.4: ARDUINO NANO BOARD

Key Features:

1. Microcontroller:

- The **Arduino Nano** is based on the **ATmega328P** microcontroller (Same as the Arduino Uno).
- The ATmega328P is an 8-bit microcontroller with 32 KB of flash memory, 2 KB of SRAM, and 1 KB of EEPROM. It can be programmed using the **Arduino IDE**.

2. Compact Size:

- The Arduino Nano is **small and breadboard-friendly**, measuring only **45mm x 18mm**, making it ideal for space-constrained projects.

3. Input and Output Pins:

- **Digital I/O pins** (of which 6 can be used as PWM outputs).
- **analog inputs** (A0 to A7).
- **PWM pins** (Digital pins 3, 5, 6, 9, 10, and 11).
- **Reset pin**.
- **Power pins**: **5V**, **3.3V**, **GND**, and **Vin** (for external voltage input). **TX/RX pins** for serial communication.

4. **Operating Voltage:**
 - **Operating voltage: 5V** (if powered via USB or external 5V). **Input voltage:** Can be powered using the **Vin pin** with a range of **712V** (recommended: 9V).
5. **USB Connectivity:**
 - The **Arduino Nano** has a built-in **USB interface** (using the **FTDI USB to-serial** chip), so you can easily connect it to your computer via a micro-USB cable for programming.
6. **Clock Speed:**
 - The **ATmega328P** microcontroller runs at a clock speed of **16 MHz**
7. **Memory:**
 - **32 KB of Flash memory** for storing the program (of which 2 KB is used by the boot loader). **2KB of SRAM** (Static RAM) for variables and data. **1 KB of EEPROM** for non-volatile storage.
8. **Onboard LEDs:**
 - **Power LED:** Indicates when the board is powered. **Pin 13 LED:** Built-in LED on digital pin 13, useful for simple testing or debugging.
9. **Programming Interface:**
 - Can be programmed via the **Arduino IDE** (Integrated Development Environment) using **USB** (FTDI interface).
 - Supports standard Arduino functions like **digital Write ()**, **analog Read ()**, **Serial. Print ()**, etc.
10. **Boot loader:**
 - The board comes pre-installed with a **boot loader**, allowing you to upload code easily via the USB connection without needing an external programmer.

Arduino Nano Pinout:

Here is a breakdown of the main pins on the **Arduino Nano**:

- **Digital I/O Pins (D0 to D13):** These pins are used for digital input/output operations. Some of them can also be used for PWM (Pulse Width Modulation) output.
- **Analog Pins (A0 to A7):** These pins can be used for analog input (e.g., reading from a sensor) or can be used as digital I/O pins.
- **Ground (GND):** The GND pins are used for connecting the ground of the circuit to the board.
- **VCC (5V):** Used for powering the board or other components requiring 5V.
- **3.3V:** Provides 3.3V regulated power.
- **Reset Pin:** Used to manually reset the Arduino.

Advantages:

- **Small Form Factor:** Its compact makes it ideal for smaller projects and for fitting into tight spaces.
- **Low Cost:** It is adorable, making it a popular choice for hobbyists and beginners.
- **Breadboard Compatibility:** The Nano's pin configuration is designed to fit perfectly on a standard breadboard for easy prototyping.

4.3.4 NEMA 17 STEPPER MOTOR:

The NEMA 17 stepper motor is one of the most commonly used stepper motors in various applications, particularly in robotics, CNC (Computer Numerical Control) machines, 3D printers, and other precise control systems. The NEMA standard refers to the National Electrical Manufacturers Association, which specifies the dimensions and mounting of motors, while the 17 refers to the flange of the motor.



Fig 4.5: Stepper Motor

Key Features and Specifications of NEMA 17 Stepper Motor:

1. Physical Dimensions:

- **Flange Size:** The motor is typically referred to as NEMA 17, where 17 refers to the flange, which is 1.7 inches (43.2 mm) square. **Length:** The length of the motor (varies depending on the specific model) usually ranges between 40mm to 60mm or even longer for high-torque variants.
- **Mounting Hole Spacing:** The motor has **four mounting holes** at a spacing of 31mm (center-to-center) for attaching to a mount or bracket.

2. Stepper Motor Type:

- **Unipolar:** Easier to control but provides less torque compared to bipolar motors.

3.Electrical Characteristics:

- **Voltage:** Typically, NEMA 17 motors operate on **2 to 12V** (depending on the specific motor).
- **Current:** The motor draws a current in the range of **1A to 2A per phase**, though this can vary based on the motor's specifications.
- **Resistance:** Resistance per phase typically ranges from **1 ohm to 5 ohms**.
- **Inductance:** Inductance of each winding can be **1 to 5 MH** (millineries).

4.Step Angle:

Standard Step Angle: The typical **step angle** is **1.8° per step**, meaning it takes 200 steps to complete one full rotation (360°). Some models may finer step angles (e.g., **0.9°**), which would result in 400 steps per revolution.

5.Holding Torque:

- The torque provided by the motor when it is stationary (holding torque) typically ranges from **30 to 80 N·cm** (Newton-centimeter). However, higher torque versions may provide up to **100 N·cm or more**.
- Holding torque is crucial in applications where the motor needs to maintain its position without losing steps.

6.Speed and Efficiency:

- **RPM (Revolutions per minute):** The typical NEMA 17 motor operates at speeds from **200 to 1200 RPM**, depending on the load and drive current.
- **Efficiency:** Stepper motors are generally efficient for controlled movements, but they are not as energy-efficient as DC motors, especially at higher speeds.

7.Applications:

- **3D Printers:** NEMA 17 motors are widely used in 3D printers to move the print head and the build platform with precision.
- **CNC Machines:** Stepper motors, including NEMA 17, are used for precise control of axes and spindles in CNC milling, laser cutting, and other machinery.
- **Robotics:** For precise motion control, such as rotating arms, wheels, and other mechanical parts.
- **Automated Systems:** Used in various automated systems that require accurate positioning and motion.
- **Conveyor Belts:** In material handling systems where precise movement of parts or products is needed.

8.Control and Drivers:

- Stepper motors require specialized **stepper motor drivers** (e.g., **A4988, DRV8825, TB6600**) to control their movements. These drivers control the current sent to the motor coils and enable precise stepping.

- **Microstepping:** Many stepper motor drivers allow for **Microstepping**, which helps to smooth the motor's movement by breaking down each step into smaller increments (e.g., 1/2 step, 1/16 step). This leads to smoother motion and reduces vibrations.

Advantages:

- **Precision and Accuracy:** Stepper motors are excellent for applications requiring precise positioning and rotation without feedback (open-loop control).
- **High Torque at Low Speeds:** They provide high torque at low speeds, making them ideal for applications like 3D printing or CNC machining.
- **Ease of Control:** Simple control schemes make them relatively easy to integrate with microcontrollers like Arduino, , and other platforms.

Limitations:

- **Energy Efficiency:** Stepper motors can be less energy-efficient compared to other types of motors like brushed DC motors. **Heat Generation:** Stepper motors can generate heat, especially when running at higher currents or under load.
- **Noise:** Stepper motors can be noisy, particularly at higher speeds or when not using Microstepping.

4.3.5 TB 6600 STEPPER MOTOR DRIVER:

The **TB6600** is a popular **stepper motor driver** used for controlling **bipolar stepper motors**. It's well-suited for applications where precise motion control is required, such as **CNC machines, 3D printers, robotics**, and other automation systems. The TB6600 is known for its **high current handling, smooth operation**, and **ease of use**, making it a great choice for driving stepper motors.



Fig 4.6: Stepper Motor Driver

Key Features of the TB6600 Stepper Motor Driver:

1. High Current Handling:

- The TB6600 driver can support stepper motors with a **current rating up to 4.5A** (per phase).
- It can be adjusted to handle a range of motor current ratings, typically from **1.5A to 4.5A** per phase, providing good compatibility with many NEMA 17 and NEMA 23 stepper motors.

2. Voltage Range:

- The TB6600 can operate with a **wide input voltage range**, typically between **9V to 42V**. This makes it flexible for different types of stepper motors and power requirements.

3. Microstepping:

- The TB6600 supports **Microstepping** operation. It can step through various microstep resolutions, including:
 1. **Full step**
 2. **Half step**
 3. **1/4 step 1/8step**
 4. **1/16 step**
- Microstepping improves the smoothness and accuracy of the motor's motion, reducing vibrations and providing finer resolution in stepper motor positioning.

4. Smooth and Quiet Operation:

- The TB6600 **smooth operation** with minimal noise compared to other stepper motor drivers, especially when used with Microstepping. This is ideal for applications that need precise and quiet movements, such as in 3D printing or CNC machines.

5. Overcurrent and Overvoltage Protection:

- The driver includes protection mechanisms for **overcurrent**, **overvoltage**, and **over temperature**, preventing damage to the driver and motor in case of adverse conditions.

6. Ease of Use:

- The TB6600 is relatively easy to use, especially with controllers like Arduino, , or other microcontrollers. It has simple connections for step, direction, and enable signals, which can be easily interfaced with your controller.

7. Wiring and Connections:

- **Step Pin (PUL):** Sends pulse signals that determine the movement of the stepper motor.
- **Direction Pin (DIR):** Controls the direction of rotation (clockwise or counterclockwise).
- **Enable Pin (ENA):** This pin enables or disables the driver.
- **Motor Wires:** The TB6600 connects to the stepper motor's coils, typically with 4 wires for a bipolar motor.

8. Heat Dissipation:

- The driver comes with an **integrated heat sink**, which helps in dissipating heat during operation, especially when driving higher current motors at higher speeds. This helps keep the TB6600 operating at a stable temperature and prevents overheating.

9. Compact Size:

- The TB6600 is **compact** enough to fit in many small enclosures, making it suitable for use in tight spaces, like in 3D printers or small CNC machines.

Applications:

- **3D Printers:** The TB6600 is commonly used in 3D printers to drive the stepper motors controlling the movement of the print head and the build platform.
- **CNC Machines:** Ideal for driving stepper motors in CNC machines, where precise movement of the tool head and axes is crucial.
- **Robotics:** Often used in robotic applications for controlling joint movements and providing accurate motion control.
- **Laser Cutters/Engravers:** Used to control stepper motors for moving laser heads or work pieces.
- **Automation Systems:** In industrial automation, the TB6600 is used for controlling stepper motors in various mechanical systems that require precision and accuracy.

4.3.6 PCA 9685 SERVO MOTOR DRIVER:

The **PCA9685** is a popular **16-channel, 12-bit PWM (Pulse Width Modulation) servo motor driver** controlled via the **I2C (Inter-Integrated Circuit) bus**. It's widely used in robotics and automation projects to control servo motors, LEDs, and other PWM-compatible devices.



Fig 4.7: SERVO MOTOR DRIVER

Key Features:

- **16-Channel PWM outputs** – can control up to **16 servos** simultaneously
- **12-bit resolution** – 4096 steps of PWM precision
- **I2C interface** – adjustable address (up to 62 PCA9685 modules on the same bus)
- **Frequency range** – Adjustable from **24 Hz to 1526 Hz**
- **Built-in clock** – No need for continuous I2C communication
- **Output voltage** – 3.3V or 5V logic compatible
- **Power supply**
- **Logic voltage:** 3.3V–5V
- **Servo power supply:** Up to 6V
- **Output type** – Open-drain with 25 mA sink capability per output
- **Built-in oscillator** – No need to rely on the controller's clock
- **PWM polarity** – Programmable (active high or low) **Output enable pin** – For fast output disabling

I2C Addressing

- Base I2C address: 0x40 (7-bit address)
- Configurable using **A0–A5** pins (can add up to 62 unique addresses)
- Example:

A0 = LOW, A1 = LOW → Address = 0x40 A0 = HIGH, A1 = HIGH → Address = 0x43

4.3.7 USB WEBCAM (Image for reference):

A USB webcam is a plug-and-play camera that connects to a computer or via a USB port for video capture and streaming.



Fig 4.8: USB Webcam

Specifications:

- **Resolution:** 720p (HD), 1080p (Full HD), 2K, 4K
- **Frame Rate:** 30 fps, 60 fps (higher for smoother video)
- **Lens:** Fixed or Autofocus
- **Microphone:** Built-in (mono or stereo)
- **Field of View (FOV):** 60°–120°
- **Interface:** USB 2.0, USB 3.0 (for higher resolution/frame rate) **Compatibility:** Windows, macOS, Linux, Android

4.3.8 DC ADAPTER 5V 1.5Amp

- **Input Voltage:** 100V–240V AC (50/60Hz)
- **Output Voltage:** 5V DC
- **Output Current:** 1.5A (Max)
- **Power Output:** 7.5W ($5V \times 1.5A$)
- **Connector:** Barrel jack (commonly 5.5mm outer, 2.1mm inner)
- **Polarity:** Center positive (+)
- **Cable Length:** ~1m (varies)
- **Protection:** Overload, overvoltage, short circuit protectio



Fig 4.9: DC ADAPTER

4.3.9 SERVO MOTOR ASSEMBLY SUPPORTING BRACKETS:

Servo Motor Assembly Supporting Brackets

Servo motor brackets are used to securely mount and position servo motors in various applications like robots, arms, and pan-tilt mechanisms.



Fig 4.10(A): **SUPPORTING BRACKETS**

Types of Servo Brackets:

1. **L-Shaped Brackets** – For basic mounting; commonly used in robotic arms.
2. **U-Shaped Brackets** – Provide additional support and flexibility for multi-axis movement.
3. **C-Shaped Brackets** – Ideal for compact setups and tight spaces.
4. **Pan-Tilt Brackets** – Allow for two-axis movement (horizontal + vertical).
5. **Adjustable Brackets** – Allow fine-tuning of servo position and angle.

Materials:

- **Aluminum** – Lightweight, strong, and corrosion-resistant.
- **Stainless Steel** – High strength and durability.
- **Plastic (ABS, Nylon)** – Lightweight but less durable.

Mounting Options:

- Pre-drilled holes for easy installation.
- Screws and nuts provided for secure attachment.
- Compatible with standard servo models (e.g., MG90S, SG90, MG995).

Applications:

1. Robotic arms
2. Camera gimbals
3. RC cars and plane
4. Humanoid robots

4.3.10 STEPPER MOTOR SUPPORTING BRACKET:

A **Stepper Motor Supporting Bracket** is a mounting accessory designed to securely hold stepper motors in place, ensuring proper alignment and stability during operation.



Fig 4.10(B): **SUPPORTING BRACKETS**

Key Features:

- **Material:** Aluminum alloy, steel, or plastic (lightweight and durable)
- **Compatibility:** Fits standard stepper motors (e.g., **NEMA 17, NEMA 23, NEMA 34**)

Design:

- Pre-drilled mounting holes for easy installation
- L-shaped or U-shaped for added strength
- Adjustable slots for fine alignment
- **Finish:** Powder-coated or anodized for corrosion resistance
- Dimensions: Vary based on motor and type

Usage:

- Used in **3D printers, CNC machines, robotics, and automation systems** Ensures vibration damping and accurate positioning.

Component	Currently Used	Future Large-Scale Component	Advantage of Future Component
Camera	USB Webcam	Industrial-Grade High-Speed Camera	Higher resolution and frame rate for fast sorting.
Controller	Arduino Nano	PLC (Programmable Logic Controller)	Robust and industrial-grade reliability.
Robotic Arm	Servo-based Robotic Arm	Air Suction Robotic Arm	Vacuum grip reduces fruit damage.
Conveyor	Fixed-Speed Conveyor	Grid Line Control Conveyor Belt	Precise tracking and control during sorting.
Processing Unit	PC + Arduino	Edge AI Device (e.g., NVIDIA Jetson)	Fast, real-time inference without the cloud.
Connectivity	Manual Logging	Smart IoT Gateway	Enables monitoring and remote analytics.
Structure	Plastic Frame	Stainless Steel Frame	More durable and hygienic.

CHAPTER 5: SOFTWARE IMPLEMENTATION

5.1 ABOUT THE SOFTWARE USED

5.1.1 Arduino IDE:

The Arduino IDE is an open-source platform used for programming Arduino microcontrollers. It provides a simple interface for writing code, uploading it to the board, and monitoring output.



Fig 5.1: Arduino IDE

Features:

- **Code Editor** – Simple text editor with syntax highlighting and auto completion.
- **Compiler** – Converts code into machine language for Arduino boards. **Serial Monitor** – Allows communication with the Arduino board to debug and test output.
- **Library Manager** – Access to a wide range of pre-built libraries for sensors, motors, displays, etc.
- **Board Manager** – Supports various Arduino boards and third-party microcontrollers.

Advantages:

- **Beginner-Friendly** – Simple and easy to learn for beginners.
- **Cross-Platform** – Works on Windows, Mac, and Linux.
- **Open-Source** – Free to use and customizable.
- **Wide Community Support** – Large community for troubleshooting and project ideas.
- **Pre-Built Libraries** – Reduces coding time with ready-to-use modules.
- **Plug-and-Play** – Easy to upload code to the board with minimal setup.

5.1.2 Python Jupyter Notebook

Jupyter Notebook is an open-source, interactive web-based environment for writing and running Python code. It allows you to create and share documents that contain:

1. Code (Python, R, Julia, etc.)
2. Visualizations (Matplotlib, Seaborn, etc.)
3. Markdown (for text, equations, and annotations)
4. Interactive Output (widgets, plots)



Fig 5.2 : Python Jupyter Notebook

Advantages of Jupyter Notebooks:

- **Interactive Coding** – Write and execute code in cells, see output instantly.
- **Visualization** – Supports inline charts and graphs for data analysis.
- **Markdown Support** – Add rich text, Latex equations, and images for better documentation.
- **Reproducibility** – Easy to share and reproduce research or analysis.
- **Flexibility** – Supports multiple programming languages (via kernels).
- **Debugging and testing** – Easy to test code line-by-line.
- **Integration** – Works well with libraries like NumPy, Pandas, TensorFlow, and Scikit-learn.

5.1.3 OPENCV & Darknet Framework

OpenCV (Open Source Computer Vision Library) Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.

Purpose: OpenCV is an open-source library for computer vision, machine learning, and image processing. Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.

Key features

- Image and video processing (e.g., filtering, edge detection, object tracking)
- Facial recognition, feature matching, and optical flow analysis
- Support for deep learning models (e.g., TensorFlow, PyTorch)

Advantages:

- Fast and optimized for real-time applications
- Cross-platform support (Windows, Linux, macOS, Android, iOS)
- Extensive documentation and active community Compatible with hardware acceleration (e.g., CUDA, OpenCL) Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.



Fig 5.3: Open CV

Key Features:

- Lightweight and fast inference
- GPU-accelerated for real-time performance
- Suitable for edge devices and embedded systems

Advantages:

- High-speed object detection (real-time at >30 FPS)
- Simple to set up and modify
- Efficient for low-power devices
- Supports custom-trained models

Darknet Framework

Darknet is an open-source neural network framework written in C and CUDA, used primarily for object detection.

Key Features:

- Lightweight and fast inference
- GPU-accelerated for real-time performance
- Suitable for edge devices and embedded systems

Advantages:

- High-speed object detection (real-time at >30 FPS)
- Simple to set up and modify
- Efficient for low-power devices
- Supports custom-trained models

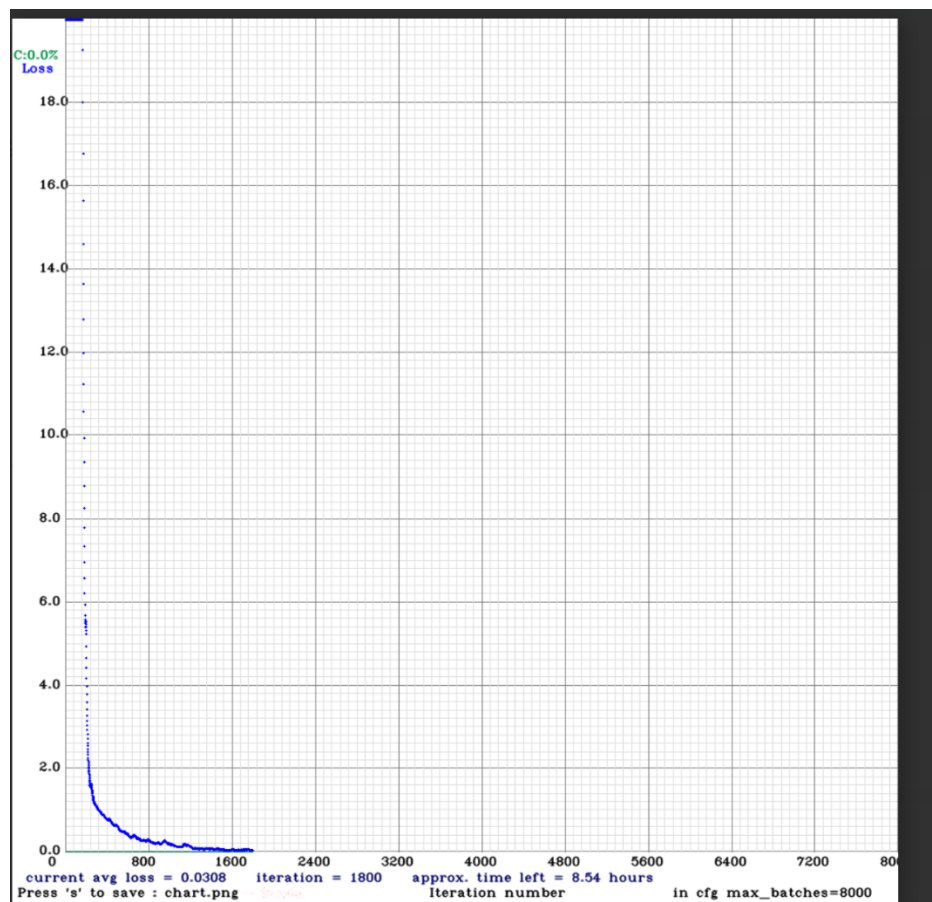


Fig 5.4: Graph

5.2 PROGRAMMING LANGUAGE USED

5.2.1 C++ :

1. C++ is a high-performance, object-oriented programming language used for system programming, game development, and real-time applications.
2. It supports low-level memory manipulation and has features like classes, inheritance, and polymorphism.

5.2.2 Python :

1. Python is a high-level, interpreted language known for its simplicity and readability, widely used in web development, data science, and AI.
2. It supports dynamic typing, automatic memory management, and extensive libraries.

5.3 CODE

```
import numpy as np
```

```
import cv2
```

```
import serial
```

```
import time
```

```
# Load YOLOv3 model and configuration
```

```
net = cv2.dnn.readNet("E:\\PROJECT\\yolov3\\yolov3.weights",  
"E:\\PROJECT\\yolov3\\yolov3.cfg")
```

```
layer_names = net.getLayerNames()
```

```
# Load class names
```

```
classes = [ ]
```

```
with open("E:\\PROJECT\\yolov3\\coco.names", "r") as f:
```

```
classes = [line.strip() for line in f.readlines()]
```

```
# Configure output layers
```

```
output_layers = net.getUnconnectedOutLayersNames()
```

```
# Open a serial connection to Arduino
```

```

arduino = serial.Serial('COM7', 9600, timeout=1)

time.sleep(2) # Wait for Arduino to initialize

# Capturing video through webcam

webcam = cv2.VideoCapture(1)

# Set up window properties

window_name = "APPLE DETECTION WINDOW"

cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)

cv2.resizeWindow(window_name, 800, 600) # Set window size

object_detected = False

command_sent = False

command_time = 0

while True:

    ret, frame = webcam.read()

    if not ret:

        break

    height, width, channels = frame.shape

    # Preprocess frame

    blob = cv2.dnn.blobFromImage(frame, 0.00392, (416, 416), (0, 0, 0), True, crop=False)

    net.setInput(blob)

    outs = net.forward(output_layers)

# Initialize variables

```


for out in outs:

for detection in out:

scores = detection[5:]

class_id = np.argmax(scores)

confidence = scores[class_id]

if confidence > 0.5 and class_id == 47: # Check if detected object is an apple

center_x = int(detection[0] * width)

center_y = int(detection[1] * height)

w = int(detection[2] * width)

h = int(detection[3] * height)

Extract region of interest (ROI) for the apple

roi = frame[max(center_y - h // 2, 0): min(center_y + h // 2, height),

max(center_x - w // 2, 0): min(center_x + w // 2, width)]

Convert ROI to HSV color space

hsv_roi = cv2.cvtColor(ro, cv2.COLOR_BGR2HSV)

Calculate total pixels in the ROI

total_pixels = hsv_roi.shape[0] * hsv_roi.shape[1]

Create color masks

**red_mask = cv2.inRange(hsv_roi, np.array([0, 100, 100]), np.array([10, 255, 255])) + **

cv2.inRange(hsv_roi, np.array([160, 100, 100]), np.array([180, 255, 255]))

```

yellow_mask = cv2.inRange(hsv_roi, np.array([20, 100, 100]), np.array([35, 255, 255]))

brown_mask = cv2.inRange(hsv_roi, np.array([10, 30, 20]), np.array([30, 100, 80]))

orange_mask = cv2.inRange(hsv_roi, np.array([10, 100, 20]), np.array([25, 255, 255]))

red_percentage = (np.count_nonzero(red_mask) / total_pixels) * 100

yellow_percentage = (np.count_nonzero(yellow_mask) / total_pixels) * 100

brown_percentage = (np.count_nonzero(brown_mask) / total_pixels) * 100

orange_percentage = (np.count_nonzero(orange_mask) / total_pixels) * 100

label = "Fruit"

if red_percentage > 5:

    label += " (Apple)"

# Draw bounding boxes and labels

font = cv2.FONT_HERSHEY_PLAIN

color = (0, 255, 0) # Green color for apple

cv2.rectangle(frame, (max(center_x - w // 2, 0), max(center_y - h // 2, 0)),

               (min(center_x + w // 2, width), min(center_y + h // 2, height)), color, 2)

cv2.putText(frame, label, (max(center_x - w // 2, 0), max(center_y - h // 2, 0) + 30), font,

2, color, 2)

# Determine if the apple is ripe or unripe and send command to Arduino

if red_percentage > 5:

    if not object_detected:

        command_time = time.time()

        object_detected = True

    if object_detected and (time.time() - command_time) >= 5 and not command_sent:

        arduino.write(b'1')

```

```

    print("Command Sent: 1")

    command_sent = True

elif confidence > 0.5 and class_id == 49: # Check if detected object is an apple

    center_x = int(detection[0] * width)

    center_y = int(detection[1] * height)

    w = int(detection[2] * width)

    h = int(detection[3] * height)


# Extract region of interest (ROI) for the apple

roi = frame[max(center_y - h // 2, 0): min(center_y + h // 2, height),

             max(center_x - w // 2, 0): min(center_x + w // 2, width)]

    # Convert ROI to HSV color space

    hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

# Calculate total pixels in the ROI

    total_pixels = hsv_roi.shape[0] * hsv_roi.shape[1]

# Create color masks

    red_mask = cv2.inRange(hsv_roi, np.array([0, 100, 100]), np.array([10, 255, 255])) + \
        cv2.inRange(hsv_roi, np.array([160, 100, 100]), np.array([180, 255, 255]))

    yellow_mask = cv2.inRange(hsv_roi, np.array([20, 100, 100]), np.array([35, 255, 255]))

    brown_mask = cv2.inRange(hsv_roi, np.array([10, 30, 20]), np.array([30, 100, 80]))

    orange_mask = cv2.inRange(hsv_roi, np.array([10, 100, 20]), np.array([25, 255, 255]))

    red_percentage = (np.count_nonzero(red_mask) / total_pixels) * 100

    yellow_percentage = (np.count_nonzero(yellow_mask) / total_pixels) * 100

    brown_percentage = (np.count_nonzero(brown_mask) / total_pixels) * 100

```

```

orange_percentage = (np.count_nonzero(orange_mask) / total_pixels) * 100

label = "Fruit"

if yellow_percentage > 0.5:

    label += " (Lemon)"

elif orange_percentage > 3:

    label += " (Orange)"

# Draw bounding boxes and labels

font = cv2.FONT_HERSHEY_PLAIN

color = (0, 255, 0) # Green color for apple

cv2.rectangle(frame, (max(center_x - w // 2, 0), max(center_y - h // 2, 0)),

               (min(center_x + w // 2, width), min(center_y + h // 2, height)), color, 2)

cv2.putText(frame, label, (max(center_x - w // 2, 0), max(center_y - h // 2, 0) + 30), font,

2, color, 2)

# Determine if the apple is ripe or unripe and send command to Arduino

if yellow_percentage > 1 or orange_percentage > 4:

    if not object_detected:

        command_time = time.time()

        object_detected = True

    if object_detected and (time.time() - command_time) >= 5 and not command_sent:

        arduino.write(b'2')

        print("Command Sent: 1")

        command_sent = True

# Check for response from Arduino

if command_sent:

```

```
response = arduino.readline().decode('utf-8').strip()

if response == 'Finished':

    print("Arduino response: Finished")

    object_detected = False

    command_sent = False

# Display output frame

cv2.imshow(window_name, frame)

if cv2.waitKey(1) & 0xFF == ord('q'):

    break

# Release resources

webcam.release()

cv2.destroyAllWindows()

arduino.close()
```

5.4 Image Processing

- The camera captures images, which are processed using OpenCV and Deep Learning (YOLOv3, YOLOv5). Additionally, the system uses the HSV color model, masking, and labeling techniques for effective fruit classification.
- The YOLOv3 model detects fruits in the frame and assigns bounding boxes with confidence scores.
- Fruits are identified based on class IDs from a trained dataset (e.g., apples, oranges, lemons, bananas).

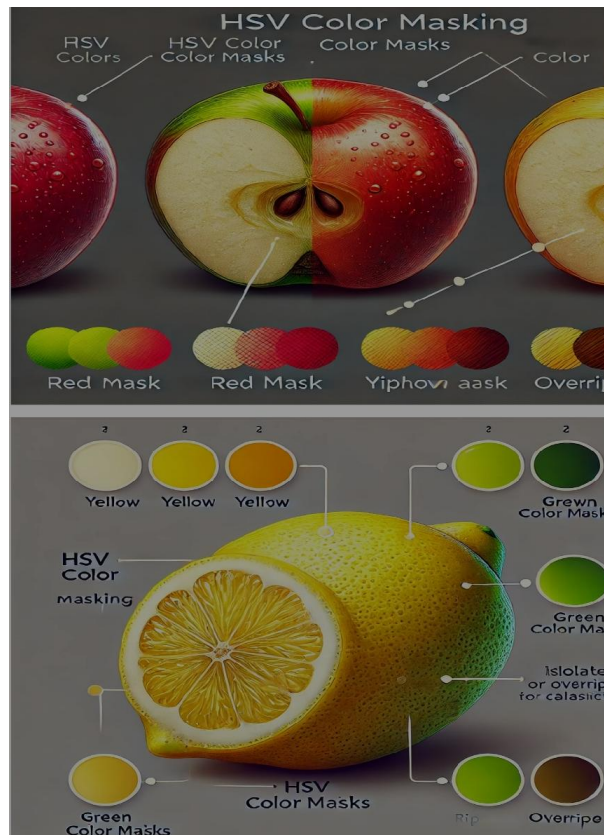


Fig 5.5 : Image Processing

5.5 Decision Making & Control Signal Processing

- Based on fruit type, , and , the system makes a decision using an AI-based classification algorithm.
- It sends commands to the robotic system (e.g., robotic arm, conveyor belt) via microcontrollers (Arduino, , or PLCs).



Fig 5.6 : Flow Chart

5.6 CODE ALGORITHM

Step 1: Open Proteus and Start a New Project

1. Open Proteus 8 Professional.
2. Click on "New Project" -> Name it "Fruit_Sorting" -> Click Next.
3. Select "Schematic Capture" as the design type.

Step 2: Add Components to the Circuit

Go to "Library" -> "Pick Devices" and search for the following components:

- Arduino Nano (Use "Arduino Nano" from the Proteus library or download it separately)
- 16x2 LCD Display (Search for "LM016L" in Proteus library) Motor Control:
- NEMA 17 Stepper Motor (Use a generic stepper motor)
- TB6600 Stepper Motor Driver (Use "L298" if TB6600 is unavailable in Proteus)
- MG996R Servo Motor (Use "Servo" from Proteus library)
- PCA9685 Servo Motor Driver (Use I2C-based PWM driver in Proteus)

Other Components:

- USB Webcam (Not simulated in Proteus, but you can use an "Input Block" to represent it)
- Power Supply (5V DC Adapter, use a "DC Power" block)

Step 3: Connect Components

1. Connect Arduino Nano to LCD Display:

- VSS -> GND
- VDD -> 5V
- RS -> D7 (or any available digital pin)
- E -> D6
- D4-D7 -> D5, D4, D3, D2
- Add a 10K potentiometer to adjust LCD contrast (Connect to VEE pin)

2. Connect TB6600 Driver to Stepper Motor:

- VCC -> 12V Power Supply
- GND -> GND
- IN1, IN2, IN3, IN4 -> Arduino Digital Pins (e.g., D8, D9, D10, D11)
- Motor Output -> NEMA 17 Stepper Motor

3. Connect PCA9685 to Arduino Nano (For Servo Control):

- VCC -> 5V
- GND -> GND
- SDA -> A4 (I2C SDA Pin of Arduino)

- SCL -> A5 (I2C SCL Pin of Arduino) - PWM Output -> MG996R Servo Motor

4. Connect Power Supply:

- 5V Power Supply -> Arduino Nano, LCD, PCA9685
- 12V Power Supply -> Stepper Motor Driver

Step 4: Add Virtual Terminal (For Debugging)

1. Go to "Instruments" -> Select Virtual Terminal.
2. Connect TX (Arduino) -> RX (Terminal), RX (Arduino) -> TX (Terminal).
3. This will display debugging messages from Arduino.

Step 5: Run the Simulation

1. Click "Compile" to check for errors.
2. Click "Run Simulation" and check:
 - LCD should display fruit classification.
 - Motors should receive signals from Arduino.
 - Virtual Terminal should show messages.

Step 6: Save the Proteus File

1. Click "File" -> "Save As".
2. Save as "FruitSorting.dsn".

CHAPTER 6: TESTING AND TROUBLESHOOTING

6.1 INITIAL STEPS WHILE TESTING

1. System Calibration

Calibrate sensors (camera, proximity, weight) and actuators (motors, grippers) for accuracy.

2. Connectivity Check

Ensure proper I2C communication and connectivity between sensors, controllers, and motors.

3. Software and Firmware Update

Load the latest software/firmware and verify compatibility with all components.

4. Dry Run Without Load

Test the robotic arm and conveyor without fruits to check movement accuracy and range.

5. Initial Load Test

Introduce sample fruits and monitor gripping, movement, and sorting accuracy.

6. Data Logging and Error Analysis

Record sensor data and system performance; identify and resolve initial errors.

6.2 PROCESS OF TESTING

INITIAL STAGE OF TESTING:-

1. Hardware Inspection and Setup

- Check physical connections of sensors, motors, grippers, and conveyor system.
- Ensure proper power supply and mechanical alignment.

2. Sensor and Vision Calibration

- Calibrate cameras, proximity, and weight sensors for accuracy.
- Test image recognition for fruit , color, and defects.

3. Motion and Control Testing

- Test robotic arm and gripper movement without load.
- Verify smooth operation of the conveyor and sorting mechanism.

4. Sample Sorting Test

- Introduce a small batch of mixed fruits.
- Evaluate sorting accuracy, speed, and handling of defective fruits.

FINAL STAGE OF TESTING:-

1. Full Load Testing

Run the system at maximum capacity with mixed fruit types to evaluate sorting speed, accuracy, and efficiency.

2. Stress and Endurance Testing

Operate the system continuously over an extended period to check for overheating, wear, and mechanical failures.

3. Performance Analysis and Optimization

Collect performance data, identify bottlenecks, and fine-tune software algorithms and mechanical settings for improved accuracy and speed.

6.3 Mid stage hardware kit

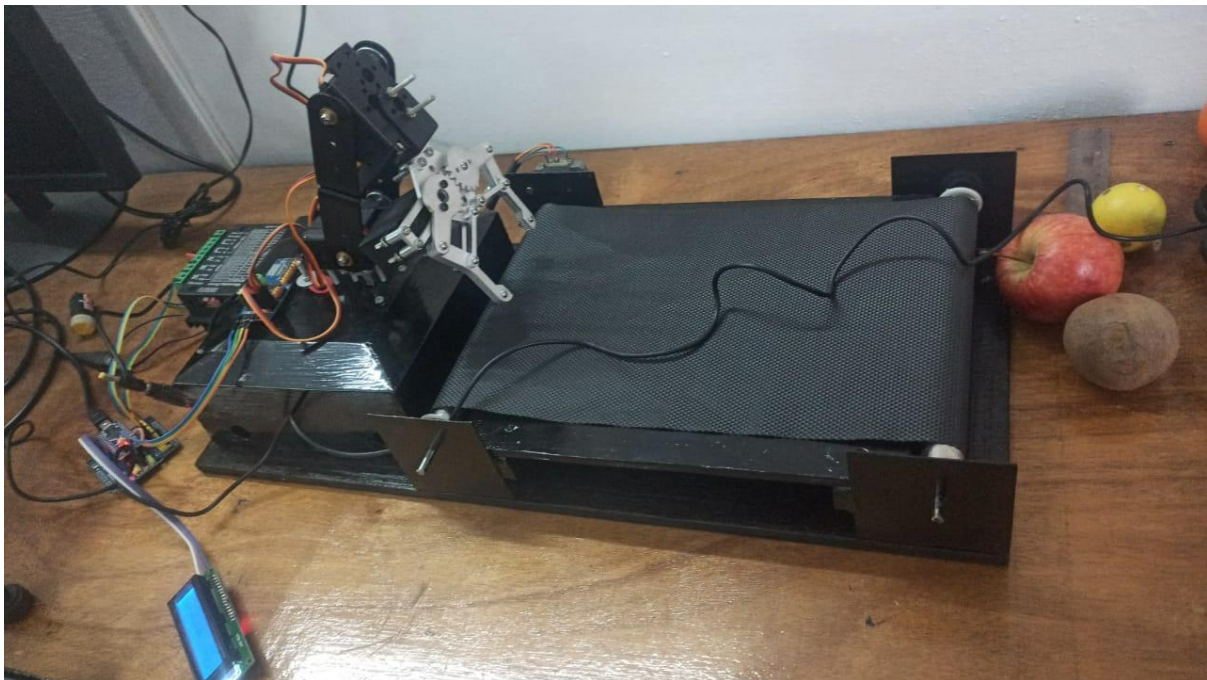


Fig 6.1(A): **hardware kit**



Fig 6.1(B): **hardware kit**

6.4 TROUBLESHOOTING

A type of problem solving called troubleshooting is frequently used to fix defective processes or goods. In order to solve the issue and restore the functionality of the process or product, a logical and methodical search is conducted to identify the problem's origin. When creating and maintaining complex systems, troubleshooting is necessary since there are numerous potential causes for the symptoms of an issue. There are many fields that require troubleshooting, including engineering, electronics, system administration, automotive repair, and diagnostic medicine. Finding the problems or symptoms in a system is the first step in troubleshooting. It's common practice to eliminate potential sources of an issue in order to determine which reason is more likely. Finally, troubleshooting requires confirmation that the solution restores the product or process to its working state. In general terms, the process of diagnosing or identifying "trouble" in a system's management flow that results from a breakdown of any kind is known as troubleshooting. Identifying and resolving the root cause of the symptoms is known as Troubleshooting.

6.5 STEPS INVOLVED IN TROUBLESHOOTING

1. **Understand the Design:** Familiarize yourself with the circuit diagram, specifications, and functionality of the electronic project. Understand the purpose of each component and how they interact with each other.
2. **Inspecting the circuit:** Inspect the circuit visually for any signs of damage or wear, such as broken wires, burnt or swollen components, loose connections, etc. It is important to check all components of the circuit, including resistors, capacitors, transistors, diodes, and any other parts.
3. **Check for Shorts and Opens:** Use a multimeter to check for short circuits (zero resistance) or open circuits (infinite resistance) between different points in the circuit. Troubleshoot and resolve any shorts or opens found.
4. **Replacing faulty components:** If any faulty components are identified, replace them with new ones of the same specifications.
5. **Check Power Supply:** Verify that the power supply to the circuit or device is functioning correctly and providing the required voltage and current. Use a multimeter to measure voltage levels at key points in the circuit.

6.6 TROUBLESHOOTING IN OUR PROJECT

1. Inaccurate Sorting

- Verify machine learning model accuracy and retrain if needed.
- Check camera alignment and lighting conditions.

2. Gripper Failure

- Inspect for mechanical damage or misalignment.
- Adjust gripping force settings based on fruit and texture.

3. Motor or Conveyor Not Working

- Check motor driver connections and power supply.
- Ensure proper PWM signal from the controller.

4. Communication Issues (I2C/Serial)

- Verify I2C addresses and bus integrity.
- Check for signal interference or loose wiring.

5. System Overload or Lag

- Optimize software for real-time processing.
- Upgrade processor or increase memory if necessary.

CHAPTER 7: RESULTS AND CONCLUSION

7.1 RESULTS

We can see that the robotic vision system successfully identifies and segregates fruits based on predefined parameters such as color, and shape . The conveyor belt transports the fruits to the designated sections according to the classification done by the system.

The robotic arm, integrated with an AI-based vision module, accurately picks and places the fruits in separate bins. The detection process updates in real-time on the monitoring display.

7.1.1 FRUIT CATEGORIZATION ON LCD MODULE

The vision system processes images and identifies the fruit type. Once detected, the classification results are shown on the LCD screen, and the robotic arm places the fruit in the corresponding section.

The application of Artificial Intelligence (AI) and Machine Learning (ML) in fruit sorting systems has shown promising results in various aspects, including:

1. **Improved Sorting Accuracy:** AI and ML-based systems, particularly using computer vision, have significantly improved the accuracy of sorting fruits based on color, shape, and other characteristics.
2. **Increased Speed and Efficiency:** Automated fruit sorting systems powered by AI and ML can process a large volume of fruits much faster than traditional manual sorting methods. This increases throughput in production lines, reducing bottlenecks and operational costs.
3. **Reduction in Waste:** The intelligent sorting systems ensure that only fruits meeting specific quality standards are selected for packaging, reducing the number of substandard products reaching consumers. This results in lower food waste, as fruits that don't meet the required standards are either discarded or redirected to secondary markets (e.g., for processing into juices, jams, or other products).
4. **Customization and Adaptability:** The AI and ML systems can be trained to adapt to different fruit types and sorting criteria. By using datasets of various fruit varieties, these systems can automatically learn the features that define the quality of each fruit, ensuring consistent results across different fruits.
5. **Cost-Effectiveness:** Although the initial investment in AI-powered sorting equipment can be high, the long-term savings in labor costs and increased productivity make these systems cost-effective. Over time, they reduce the reliance on manual labor and minimize human errors, leading to better returns on investment.

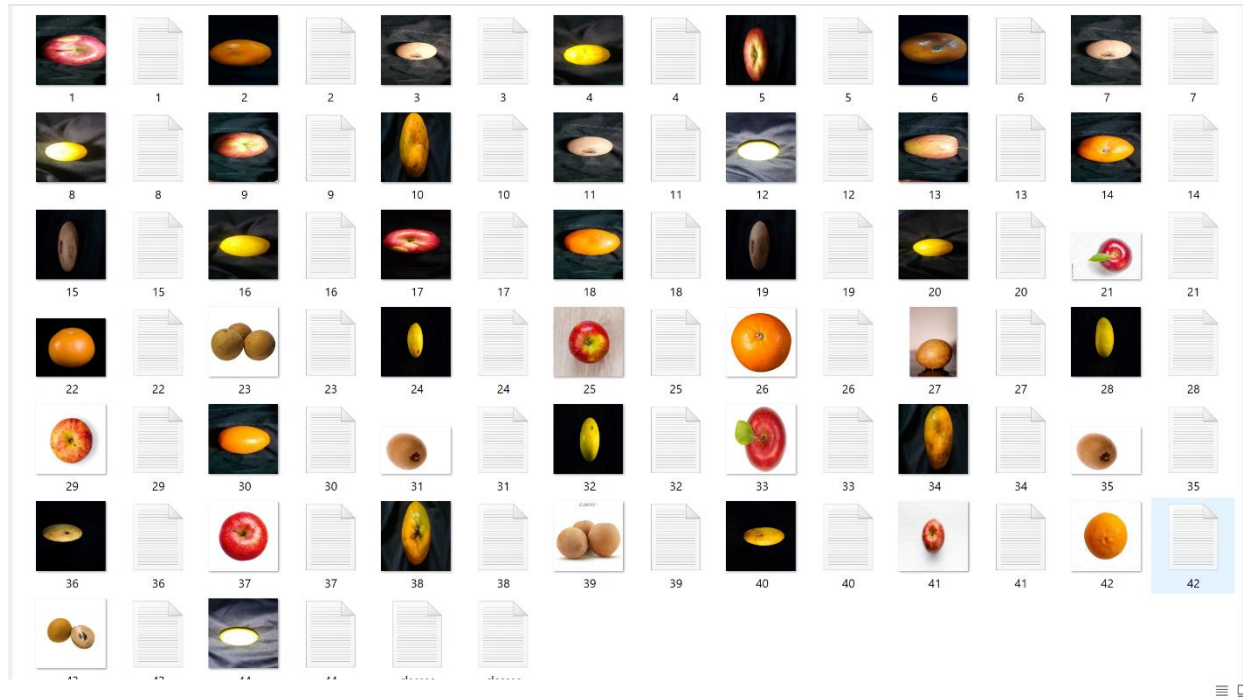


Fig 7.1: custom.names file

7.2 CONCLUSION

The integration of AI and ML technologies in fruit sorting systems is revolutionizing the agricultural industry, offering clear benefits in terms of efficiency, accuracy, and sustainability. These technologies enable higher levels of automation that optimize the sorting process, ensuring better quality control and reducing waste. As AI and ML continue to evolve, the future of fruit sorting systems is expected to be even more advanced, with deeper learning capabilities, real-time data analysis, and even better customization for a wider range of fruit types. In the long run, this will lead to a more sustainable and profitable fruit supply chain, benefiting both producers and consumers alike. However, challenges like the initial setup costs, data collection, and system maintenance still need to be addressed. Despite this, the potential benefits far outweigh the challenges, making AI and ML technologies a highly promising solution for modern fruit sorting operations.

With the increasing demand for efficient and accurate fruit sorting in agricultural and industrial sectors, manual segregation methods prove to be time-consuming and inconsistent. The proposed robotic vision system provides an automated, digital solution to enhance the efficiency of fruit classification. This system identifies, classifies, and segregates fruits using an AI-driven vision module and a robotic arm, which minimizes human intervention and ensures precision. The use of infrared and image processing sensors enables accurate detection of fruit characteristics such as color and defects. The implementation of this automated segregation system can significantly reduce labor costs and improve sorting accuracy. Additionally, this system is adaptable for use in various agricultural sectors, packaging industries, and food processing units.

CHAPTER 8: FUTURE SCOPE AND APPLICATION

8.1 FUTURE SCOPE

Future research in automated fruit sorting is likely to focus on improving the accuracy, speed, and robustness of models while addressing existing challenges. Key areas for further exploration include:

- **Hybrid Models:** Combining s with other techniques, such as reinforcement learning or unsupervised learning, to improve defect detection and classification of new fruit varieties .
- **Multi-modal Sensing:** Integrating visual, tactile, and spectral sensors (e.g., NIR or hyperspectral imaging) to improve fruit quality inspection and internal defect detection.
- **Edge AI:** Further development of edge computing solutions to enable real-time, low-latency fruit sorting in resource-constrained environments.

8.2 APPLICATION

8.2.1. Improving Processing Efficiency in Juice and Canning Industries

Waste Reduction: By accurately identifying fruits that cannot be sold fresh (due to defects or over-), the system minimizes waste. This reduces losses in the food production process and ensures a more efficient use of available resources.

8.2.2. Sustainability and Waste Reduction

Minimizing Energy and Resources: By automating the sorting process, these systems reduce the need for manual labor, which can be energy- and resource-intensive. More efficient sorting and grading systems also minimize energy use in warehouses, storage, and transportation by ensuring better inventory management.

8.2.3. Integration with Internet of Things (IoT)

- **Smart Farm-to-Table Solutions:** Sorting systems can be integrated with IoT devices to track the freshness and conditions of fruits throughout the entire supply chain. From the farm to the consumer's table, this system ensures that fruits are stored, transported, and sorted under optimal conditions. IoT devices can monitor variables like temperature, humidity, and gas concentration in storage areas to ensure fruit quality is maintained.
- **Remote Monitoring and Control:** With IoT integration, operators can monitor and control the sorting process remotely, receiving real-time alerts and updates. This leads to more proactive management of sorting and distribution.

8.2.4. High-Volume Fruit Sorting for Large-Scale Operations

- **Industrial Scale Operations:** Automated fruit sorting systems are especially beneficial for large-scale fruit producers who need to process hundreds or thousands of tons of fruit each day. These systems can handle large volumes with speed and accuracy, minimizing labor costs and ensuring that fruits are sorted efficiently and accurately at scale.
- **Increased Throughput:** With AI and ML models improving the sorting decision-making process, fruit sorting systems can process higher volumes of fruit per minute

compared to manual sorting methods, leading to improved throughput and reduced operational delays.

➤ **LIST OF EACH COMPONENT COST**

8.3 COMPONENT COST BREAKDOWN

Component	Current Component Cost (INR)	Future Component Cost (INR)
USB Webcam	800	15,000 (Industrial Camera)
Arduino Nano	400	8,000 (PLC Controller)
Servo Robotic Arm	2,000	45,000 (Industrial Arm)
Fixed Conveyor	5,000	20,000 (Variable Speed Conveyor)
PC + Arduino	25,000	30,000 (Edge AI Device)
Manual Logging	0	5,000 (Smart IoT Gateway)
Plastic Frame	2,000	10,000 (Stainless Steel Frame)
Total	35,200	1,33,000

REFERENCES

- 1.M. Kumar, A. Singh, and R. Sharma, "Vision-Based Fruit Sorting and Grading System Using Machine Learning," *2021 International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE)*. Available at: [DOI: 10.1109/ICSTCEE49637.2021.9410572](https://doi.org/10.1109/ICSTCEE49637.2021.9410572)
- 2.S. Kumar et al., "Computer Vision-Based Prototype Robotic Picking Cum Grading System," *2023 ScienceDirect*. Available at: [ScienceDirect](https://www.sciencedirect.com)
- 3.R. S. Ramkumar et al., "Machine Vision-Based Automated 3-DOF Articulated Robot for Fruit Defect Identification and Segregation," *ResearchGate*. Available at: [ResearchGate](https://www.researchgate.net)
- 4.D. M. Bulanon et al., "Development of a Visual Servo System for Robotic Fruit Harvesting," *MDPI Sensors*. Available at: MDPI
- 5.Y. Lin et al., "Recognition and Localization Methods for Vision-Based Fruit Picking Robots," *Frontiers in Plant Science*. Available at: Frontiers
- 6.Y. Zhang et al., "Intelligent Robotics Harvesting System Process for Fruits Grasping Using Deep Learning," *Nature Scientific Reports*. Available at: [Nature](https://www.nature.com)
- 7.L. Wang et al., "Key Technologies for Machine Vision for Picking Robots: Review and Prospect," *Springer Link*. Available at: [Springer Link](https://www.springer.com)
- 8.H. Kang and C. Chen, "Fruit Detection, Segmentation, and 3D Visualization in Apple Orchards," *arXiv*. Available at: [arXiv](https://arxiv.org)
- 9.T. T. Santos et al., "Grape Detection, Segmentation, and Tracking Using Deep Neural Networks," *arXiv*. Available at: [arXiv](https://arxiv.org)
- 10.K. Zhang et al., "Active Laser-Camera Scanning for High-Precision Fruit Localization in Robotic Harvesting," *arXiv*. Available at: [arXiv](https://arxiv.org)
- 11.M. Pérez-Ruiz et al., "Vision Systems for Harvesting Robots: Produce Detection and Localization," *ScienceDirect*. Available at: [ScienceDirect](https://www.sciencedirect.com)
- 12.J. Blasco et al., "A Vision System Based on a Laser Range-Finder Applied to Robotic Fruit Harvesting," *ResearchGate*. Available at: [ResearchGate](https://www.researchgate.net)
- 13.X. Xiang et al., "Three-Dimensional Location Methods for the Vision System of a Fruit-Harvesting Robot," *SpringerLink*. Available at: [SpringerLink](https://www.springer.com)
- 14.M. Patil et al., "IoT Based Economic Smart Vehicle Parking System," *IEEE Xplore*. Available at: IEEE Xplore
- 15.Pooja Ajmera, "A Review Paper on Infrared Sensor," *International Journal of Engineering Research and Technology (IJERT)*. Available at: IJERT
- 16.Kiran J. Dhande et al., "Automated Car Parking System," *International Journal of Emerging Trends in Science, Technology, and Management*. Available at: IJETSTM
- 17.A. C. Bento et al., "An Experiment with NRF24L01+ and Arduino Pro Micro Data Transmission for IoT," *IEEE Xplore*. Available at: IEEE Xplore
- 18.Baishali De and Dr. Anirban Karmakar, "Arduino Based RF Compact Module for Short Range Wireless Communication Using nRF24L01," *IJERT*. Available at: IJERT

19.V. Sumathi et al., "Energy Efficient Automated Car Parking System," *International Journal of Engineering and Technology (IJET)*. Available at: IJET

20.S. Karthik and K. Sathish, "Agricultural Robot for Fruit Picking Using Machine Vision," *Springer Link*. Available at: [SpringerLink](#)